



# Advanced Web Programming

Ung Văn Giàu  
Email: [giau.ung@eiu.edu.vn](mailto:giau.ung@eiu.edu.vn)



# State Management

- We will build a simple app containing a header that displays the user's name, with the main content also referencing the user's name.
- The user's name will be stored in state that needs to be accessed by several components.

# Topic

- Creating the project
- Using prop drilling
- Using React context
- Using Redux



# Technical requirements

- Node.js and npm
- Visual Studio Code



# Creating the project

- Create a React and TypeScript project
- Install and configure Tailwind



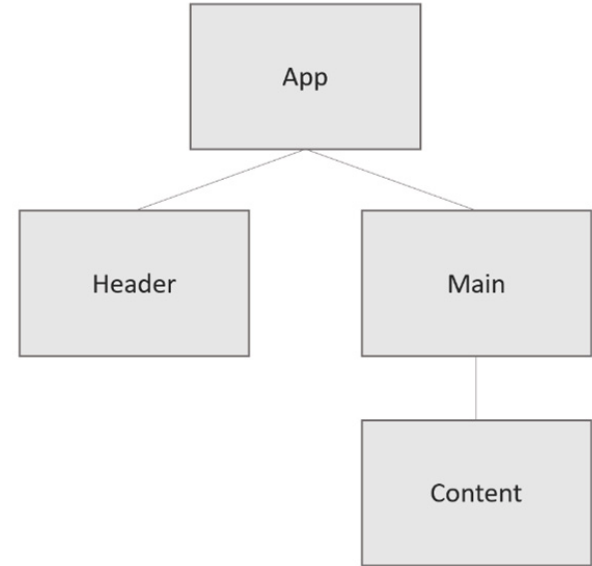
# Creating the project

- The app will contain a header and some content beneath it.
- Here is the component structure:

The header will have a **Sign in** button to authenticate and authorize a user to get their name and permissions.

Once authenticated, the **user's name** will be displayed in the app header, and the user will be welcomed in the content.

If the user has **admin permissions**, important **content** will be shown.



# Creating the project

- Carry out the following steps:

- Start by creating a file containing a function to authenticate a user.

Create a folder called **api** in the **src** folder. Then, create a file called **authenticate.ts** in the **api** folder:

```
export type User = {
  id: string;
  name: string;
};
export function authenticate(): Promise<User | undefined>
{
  return new Promise((resolve) =>
    {
      setTimeout(() => resolve({ id: "1", name: "Bob" }),
        1000)
    }
  );
}
```



# Creating the project

- Carry out the following steps:
  - Create a file containing a function to authorize a user.

Create a file called **authorize.ts** in the **api** folder

```
export function authorize(id: string): Promise<string[]>
{
  return new Promise((resolve) =>
    setTimeout(() => resolve(["admin"]), 1000)
  );
}
```

# Creating the project

- Carry out the following steps:
  - Create a component for the app header.

Create a file called **Header.tsx** in the **src** folder

```
import { User } from '../api/authenticate';

type Props = {
  user: undefined | User;
  onSignInClick: () => void;
  loading: boolean;
};
```

# Creating the project

- Carry out the following steps:
  - Add the following component implementation into **Header.tsx**:

```
export function Header({
  user,
  onSignInClick,
  loading,
}: Props) {
  return (
    <header className="flex justify-between items-center
      border-b-2 border-gray-100 py-6">
      {user ? (
        <span className="ml-auto font-bold">
          {user.name} has signed in
        </span>
      ) : (
        <button
          onClick={onSignInClick}
          className="whitespace-nowrap inline-flex items-
            center justify-center ml-auto px-4 py-2 w-36
            border border-transparent rounded-md
            shadow-sm text-base font-medium text-white
            bg-indigo-600 hover:bg-indigo-700"
          disabled={loading}
        >
          {loading ? '...' : 'Sign in'}
        </button>
      )}
    </header>
  );
}
```

# Creating the project

- Carry out the following steps:
  - Create a file called **Main.tsx** in the **src** folder

```
import { User } from './api/authenticate';
import { Content } from './Content';
type Props = {
  user: undefined | User;
  permissions: undefined | string[];
}
```

# Creating the project

- Carry out the following steps:
  - Add the following component implementation in **Main.tsx**:

```
export function Main({ user, permissions }: Props) {  
  return (  
    <main className="py-8">  
      <h1 className="text-3xl text-center font-bold  
        underline">Welcome</h1>  
      <p className="mt-8 text-xl text-center">  
        {user ? `Hello ${user.name}!` : "Please sign in"}  
      </p>  
      <Content permissions={permissions} />  
    </main>  
  );  
}
```

# Creating the project

- Carry out the following steps:
  - The last file to create in the **src** folder is called **Content.tsx**

```
type Props = {
  permissions: undefined | string[];
};
export function Content({ permissions }: Props) {
  if (permissions === undefined) {
    return null;
  }
  return permissions.includes('admin') ? (
    <p className="mt-4 text-l text-center">
      Some important stuff that only an admin can do
    </p>
  ) : (
    <p className="mt-4 text-l text-center">
      Insufficient permissions
    </p>
  );
}
```

# Using prop drilling

- In this first state management approach, we will store the **user**, **permissions**, and **loading** state in the **App** component. The **App** component will then pass this state to the **Header** and **Main** components using props.
- This approach is referred to as **prop drilling** because the state is passed down the component tree using props.

# Using prop drilling

- Rework the **App** component to store the **user**, **permissions**, and **loading** state, and pass this state down to the **Header** and **Main** components:
- Carry out the following steps:
  - Open **App.tsx**, remove all the existing code and add the following **import** statements:

```
import { useReducer } from 'react';  
import { Header } from '../Header';  
import { Main } from '../Main';  
import { authenticate, User } from '../api/authenticate';  
import { authorize } from '../api/authorize';
```



# Using prop drilling

- Carry out the following steps:
  - After the **import** statements, add a type for the state and create a variable for the initial state values.

```
type State = {  
  user: undefined | User,  
  permissions: undefined | string[],  
  loading: boolean,  
};  
const initialState: State = {  
  user: undefined,  
  permissions: undefined,  
  loading: false,  
}
```

# Using prop drilling

- Carry out the following steps:
  - Create a type for the different actions that can update the state:

```
type Action =  
  | {  
    type: "authenticate",  
  }  
  | {  
    type: "authenticated",  
    user: User | undefined,  
  }  
  | {  
    type: "authorize",  
  }  
  | {  
    type: "authorized",  
    permissions: string[],  
  };
```

# Using prop drilling

- Carry out the following steps:
  - Add a **reducer** function that updates the state:

```
function reducer(state: State, action: Action): State {  
  switch (action.type) {  
    case "authenticate":  
      return { ...state, loading: true };  
    case "authenticated":  
      return { ...state, loading: false, user: action.  
        user };  
    case "authorize":  
      return { ...state, loading: true };  
      case "authorized":  
    return {  
      ...state,  
      loading: false,  
      permissions: action.permissions,  
    };  
    default:  
      return state;  
  }  
}
```

# Using prop drilling

- Carry out the following steps:
  - Define the **App** component.

```
function App() {  
  const [{ user, permissions, loading }, dispatch] =  
    useReducer(reducer, initialState);  
  
  return (  
    <div className="max-w-7xl mx-auto px-4">  
      <Header  
        user={user}  
        onSignInClick={handleSignInClick}  
        loading={loading}  
      />  
      <Main user={user} permissions={permissions} />  
    </div>  
  );  
}  
  
export default App;
```

# Using prop drilling

- Carry out the following steps:
  - The **Header** element in the JSX references a handler called **handleSignInClick**, which needs implementation. Create this above the return statement.
  - Run the app: **npm run dev**.
  - Click the **Sign in** button. The authentication and authorization processes then happen.

```
async function handleSignInClick() {
  dispatch({ type: "authenticate" });
  const authenticatedUser = await authenticate();
  dispatch({
    type: "authenticated",
    user: authenticatedUser,
  });
  if (authenticatedUser !== undefined) {
    dispatch({ type: "authorize" });
    const authorizedPermissions = await authorize(
      authenticatedUser.id
    );
    dispatch({
      type: "authorized",
      permissions: authorizedPermissions,
    });
  }
}
```

# Using prop drilling

Remark

- This approach is simple and uses React features we are already familiar with.
  - A downside of this approach is that it forces all components between the component providing state and components accessing the state to have a prop for that state.
- Some components that do not need access to the state are forced to access it.

# Using prop drilling

## Recap

It is fine to share state across a few adjacent components using props but isn't ideal for sharing across lots of components far apart in the component tree.



# Using React context

## Understanding React context

- **React context** is an **object that can be accessed by components**. This object can contain state values, so it provides a **mechanism for sharing state across components**.
- A context is created using a **createContext** function as follows:  

```
const SomeContext = createContext<ContextType>(defaultValue);
```
- A **default value** for the context **must be passed** into **createContext**. It also has a generic type parameter for the type that represents the object created by **createContext**.



# Using React context

## Understanding React context

- The context also contains a **Provider** component that needs to **be placed above components requiring access to the context object** in the component tree.
- A provider wrapper component can be created that stores the shared state and passes it to the context Provider component as follows:

```
export function SomeProvider({ children }: Props) {  
  const [someState, setSomeState] = useState(initialState);  
  return (  
    <SomeContext.Provider value={{ someState }}>  
      {children}  
    </SomeContext.Provider>  
  );  
}
```

# Using React context

## Understanding React context

- The provider wrapper component can then be placed appropriately in the component tree, above components requiring the shared state:

```
function App() {  
  return (  
    <SomeProvider>  
      <Header />  
      <Main />  
    </SomeProvider>  
  );  
}
```

# Using React context

## Understanding React context

- React also contains a **useContext** hook that can be used so that **the context values can be consumed** as a hook, as follows:  

```
const { someState } = useContext(SomeContext);
```
- The context must be passed into `useContext` and properties from the context object can be destructured from its result.

# Using React context

## Understanding React context

- Components that want access to the shared state can access it using **useContext** as follows:

```
export function SomeComponent() {  
  const { someState } = useContext(SomeContext);  
  return <div>I have access to {someState}</div>;  
}
```

# Using React context

## Using React context

- **Description:** We will refactor the app from the last section to use React context. We will start by creating a file containing the context and the provider wrapper. Then, we will use **useReducer** in the provider wrapper to store the state. We will also create a wrapper for **useContext** to make consuming it easy.
- Carry out the following steps:
  - Creating a file called **AppContext.tsx** in the **src** folder. This will contain the context, the provider wrapper, and the **useContext** wrapper.

# Using React context

## Using React context

- Carry out the following steps:
  - Add the following import statements to **AppContext.tsx**:

```
import {  
  createContext,  
  useContext,  
  useReducer,  
  ReactNode,  
} from 'react';  
import { User } from '../api/authenticate';
```

# Using React context

## Using React context

- Carry out the following steps:
  - Add a type for the state and a variable for the initial state values.

We already have these in **App.tsx**, so the following lines can be moved from **App.tsx** to **AppContext.tsx**:

```
type State = {  
  user: undefined | User,  
  permissions: undefined | string[],  
  loading: boolean,  
};  
  
const initialState = {  
  user: undefined,  
  permissions: undefined,  
  loading: false,  
};
```

# Using React context

## Using React context

- Carry out the following steps:
  - The **Action** type and the reducer function can be moved from **App.tsx** to **AppContext.tsx**.

```
type Action =
| {
  type: "authenticate",
}
| {
  type: "authenticated",
  user: User | undefined,
}
| {
  type: "authorize",
}
| {
  type: "authorized",
  permissions: string[],
};

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "authenticate":
      return { ...state, loading: true };
    case "authenticated":
      return { ...state, loading: false, user: action.user };
    case "authorize":
      return { ...state, loading: true };
    case "authorized":
      return { ...state, loading: false, permissions: action.permissions };
    default:
      return state;
  }
}
```



# Using React context

## Using React context

- Carry out the following steps:
  - Create a type for the context in **AppContext.tsx**:

```
type AppContextType = State & {  
  dispatch: React.Dispatch<Action>  
};
```

# Using React context

## Using React context

- Carry out the following steps:
  - Create the context

```
const AppContext = createContext<AppContextType>({  
  ...initialState,  
  dispatch: () => {},  
});
```

# Using React context

## Using React context

- Carry out the following steps:
  - Implement the provider wrapper

```
type Props = {
  children: ReactNode;
};

export function AppProvider({ children }: Props) {
  const [{ user, permissions, loading }, dispatch] =
    useReducer(reducer, initialState);

  return (
    <AppContext.Provider
      value={{
        user,
        permissions,
        loading,
        dispatch,
      }}
    >
      {children}
    </AppContext.Provider>
  );
}
```

# Using React context

## Using React context

- Carry out the following steps:
  - The last thing to do in **AppContext.tsx** is to create a wrapper for **useContext**.

```
export const useAppContext = () =>  
  useContext(AppContext);
```

# Using React context

## Using React context

- Make the necessary changes to the **App**, **Header**, **Main**, and **Content** components to access the user and permissions information from **AppContext**
- Carry out the following steps:
  - Open **Header.tsx**, import the **authenticate**, **authorize**, and **useAppContext** functions. Also, remove the **User** type and the props for the **Header** component:

```
import { authenticate } from '../api/authenticate';
import { authorize } from '../api/authorize';
import { useAppContext } from '../AppContext';

export function Header() {
  return ...
}
```

# Using React context

## Using React context

- Carry out the following steps:
  - **Header** will now handle the sign-in process instead of App. So, move the **handleSignInClick** handler from App in **App.tsx** to **Header.tsx** and place it above the return statement

```
export function Header() {  
  async function handleSignInClick() {  
    dispatch({ type: 'authenticate' });  
    const authenticatedUser = await authenticate();  
    dispatch({  
      type: 'authenticated',  
      user: authenticatedUser,  
    });  
    if (authenticatedUser !== undefined) {  
      dispatch({ type: 'authorize' });  
      const authorizedPermissions = await authorize(  
        authenticatedUser.id  
      );  
      dispatch({  
        type: 'authorized',  
        permissions: authorizedPermissions,  
      });  
    }  
  }  
  return ...  
}
```

# Using React context

## Using React context

- Carry out the following steps:
  - Update the **sign-in** click handler to reference the function we just added:

```
<button
  onClick={handleSignInClick}
  className=...
  disabled={loading}
>
  {loading ? '...' : 'Sign in'}
</button>
```

# Using React context

## Using React context

- Carry out the following steps:
  - The last thing to do in **Header.tsx** is to get **user**, **loading**, and **dispatch** from the context. Add the following call to **useAppContext** at the top of the component:

```
export function Header() {  
  const { user, loading, dispatch } = useAppContext();  
  ...  
}
```



# Using React context

## Using React context

- Carry out the following steps:
  - Open **Main.tsx**, remove the import statement for the **User** type and add an import statement for **useAppContext**:

```
import { Content } from './Content';  
import { useAppContext } from './AppContext';
```

# Using React context

## Using React context

- Carry out the following steps:
  - Remove the props for the **Main** component and get user from **useAppContext**:

```
export function Main() {  
  const { user } = useAppContext();  
  return ...  
}
```

- In the JSX in **Main**, remove the **permissions** attribute on the **Content** element:

```
<Content />
```

# Using React context

## Using React context

- Carry out the following steps:
  - Open **Content.tsx** and add an import statement for **useAppContext**:
- Remove the props for the **Content** component and get **permissions** from **useAppContext**:

```
export function Content() {  
  const { permissions } = useAppContext();  
  if (permissions === undefined) {  
    return null;  
  }  
  return ...  
}
```

# Using React context

## Using React context

- Carry out the following steps:
  - Open **App.tsx**, remove the import statements except for **Header** and **Main**, and add an import statement for **AppProvider**:

```
import { Header } from './Header';  
import { Main } from './Main';  
import { AppProvider } from './AppContext';
```

# Using React context

## Using React context

- Carry out the following steps:
  - Still in **App.tsx**, remove the call to **useReducer** and remove all the attributes passed to **Header** and **Main**:

```
function App() {  
  return (  
    <div className="max-w-7xl mx-auto px-4">  
      <Header />  
      <Main />  
    </div>  
  );  
}
```

# Using React context

## Using React context

- Carry out the following steps:
  - Wrap **AppProvider** around **Header** and **Main** so that they can access the context:

```
function App() {  
  return (  
    <div className="max-w-7xl mx-auto px-4">  
      <AppProvider>  
        <Header />  
        <Main />  
      </AppProvider>  
    </div>  
  );  
}
```

# Using React context

## Recap

- In comparison to prop drilling, React context requires more code to be written.
- However, it allows components to access shared state using a hook rather than passing it through components using props.
- It's an elegant, shared-state solution, particularly when many components share state.

# Using Redux

## Understanding Redux

- Redux is a mature state management library that was first released in 2015.
- It was **released before React context** and became a **popular approach for shared state management**.



# Using Redux

## Understanding Redux

### Creating a store

- In Redux, the **state lives in a centralized immutable object** referred to as a **store**.  
There is **only a single store for the whole app**.
- Like `useReducer`, the state in a store is updated by **dispatching an action**, which is an object containing the type of change and any data required to make the change.
- An **action is handled** by a **reducer** function, which **creates a new version of the state**.

# Using Redux

## Understanding Redux

### Creating a store

- A Redux store can be created using the Redux Toolkit's **configureStore** function as follows:

```
export const store = configureStore({  
  reducer: {  
    someFeature: someFeatureReducer,  
    anotherFeature: anotherFeatureReducer  
  },  
});
```

- The **configureStore** function takes in the **store's reducers**. Each feature in the app can have its own area of state and reducer to change the state. The different areas of state are often referred to as **slices**.

# Using Redux

## Understanding Redux

### Creating a store

- The Redux Toolkit has a function to create slices, called **createSlice**.

The **createSlice** function takes in an **object** parameter containing the slice **name**, the **initial state**, and **functions to handle the different actions and update the state**.

```
export const someSlice = createSlice({
  name: "someFeature",
  initialState,
  reducers: {
    someAction: (state) => {
      state.someValue = "something";
    },
    anotherAction: (state) => {
      state.someOtherValue = "something else";
    },
  },
});
```

# Using Redux

## Understanding Redux

### Creating a store

- The **slice** created from **createSlice** contains a **reducer** function that wraps the action handlers. This **reducer** function can be referenced in the reducer property of **configureStore** when the store is created:

```
export const store = configureStore({  
  reducer: {  
    someFeature: someSlice.reducer,  
    ...  
  },  
});
```

# Using Redux

## Understanding Redux

### Providing the store to React components

- The Redux store is defined in the component tree using its **Provider** component.
- The **value** of the Redux store (from **configureStore**) needs to be specified on the **Provider** component.
- The **Provider** component must be placed above the components requiring access to the store:

```
<Provider store={store}>  
  <SomeComponent />  
  <AnotherComponent />  
</Provider>
```

# Using Redux

## Understanding Redux

### Accessing the store from a component

- Components can access state from the Redux store using a **useSelector** hook from React Redux.
- A function that selects the relevant state in the store is passed into **useSelector**:

```
const someValue = useSelector(  
  (state: RootState) => state.someFeature.someValue  
);
```

# Using Redux

## Understanding Redux

### Dispatching actions to the store from a component

- React Redux also has a **useDispatch** hook that returns a **dispatch** function that can be used to dispatch actions.
- The action is a function from the slice created using **createSlice**:

```
const dispatch = useDispatch();  
return (  
  <button onClick={() => dispatch(someSlice.actions.  
someAction())}>  
    Some button  
  </button>  
) ;
```

# Using Redux

## Install Redux

Install Redux and the Redux Toolkit into our project by the following command:

```
npm i @reduxjs/toolkit react-redux
```



# Using Redux

## Using Redux

### Description:

We can refactor the app to use Redux instead of React context.

- First, we will create a **Redux slice** for the user information before creating a **Redux store** with this slice.
- We will then move on to **add the store** to the React component tree and **consume** it in the Header, Main, and Content components.

# Using Redux

## Creating a Redux Slice

- Carry out the following steps:
  - Create a folder called **store** in the **src** folder and then a file called **userSlice.ts** within it.
  - Add the following import statements to **userSlice.ts**:

```
import { createSlice } from '@reduxjs/toolkit';  
import type { PayloadAction } from '@reduxjs/toolkit';  
import { User } from '../api/authenticate';
```

# Using Redux

## Creating a Redux Slice

- Carry out the following steps:
  - Copy the following **State** type and **initial** state value from **AppContext.tsx** into **userSlice.ts**:

```
type State = {  
  user: undefined | User;  
  permissions: undefined | string[];  
  loading: boolean;  
};  
  
const initialState: State = {  
  user: undefined,  
  permissions: undefined,  
  loading: false,  
};
```

# Using Redux

## Creating a Redux Slice

- Carry out the following steps:
  - Create the slice in **userSlice.ts**

```
export const userSlice = createSlice({  
  name: 'user',  
  initialState,  
  reducers: {  
  
  }  
});
```

# Using Redux

## Creating a Redux Slice

- Carry out the following steps:
  - Define the following action handlers inside the **reducers** object:

```
reducers: {  
  authenticateAction: (state) => {  
    state.loading = true;  
  },  
  authenticatedAction: (  
    state,  
    action: PayloadAction<User | undefined>  
  ) => {  
    state.user = action.payload;  
    state.loading = false;  
  },  
  authorizeAction: (state) => {  
    state.loading = true;  
  },  
  authorizedAction: (  
    state,  
    action: PayloadAction<string[]>  
  ) => {  
    state.permissions = action.payload;  
    state.loading = false;  
  }  
}
```

# Using Redux

## Creating a Redux Slice

- Carry out the following steps:
  - Export the **action handlers** and the **reducer** function from the slice:

```
export const {  
  authenticateAction,  
  authenticatedAction,  
  authorizeAction,  
  authorizedAction,  
} = userSlice.actions;  
  
export default userSlice.reducer;
```

# Using Redux

## Creating the Redux store

- Carry out the following steps:
  - Create a file called **store.ts** in the **store** folder containing the following import statements:

```
import { configureStore } from '@reduxjs/toolkit';  
import userReducer from '../userSlice';
```

# Using Redux

## Creating the Redux store

- Carry out the following steps:
  - Use the **configureStore** function to create the store referencing the reducer from the slice we created earlier:

```
export const store = configureStore({  
  reducer: { user: userReducer }  
});
```



# Using Redux

## Creating the Redux store

- Carry out the following steps:
  - The last thing to do in **store.ts** is export the type for Redux's full state object, which we will eventually require in the **useSelector** hook in components consuming the Redux store:

```
export type RootState = ReturnType<typeof store.getState>;
```

- ✓ **ReturnType** is a standard TypeScript utility type that returns the return type of the function type passed into it.
- ✓ The **getState** function in the Redux store returns the full state object.

# Using Redux

## Adding the Redux store to the component tree

- Carry out the following steps:
  - Open **App.tsx** and remove the **AppContext** import statement. Remove the **AppContext.tsx** file as well because this is no longer required.
  - Add an import statement for the **Provider** component from React Redux and the Redux store we created:

```
import { Provider } from 'react-redux';  
import { store } from '../store/store';
```

# Using Redux

## Adding the Redux store to the component tree

- Carry out the following steps:
  - Replace **AppProvider** with **Provider** in the JSX

```
<div className="max-w-7xl mx-auto px-4">  
  <Provider store={store}>  
    <Header />  
    <Main />  
  </Provider>  
</div>
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Open **Header.tsx** and remove the **AppContext** import statement.
  - Add the following import statements to **Header.tsx**:

```
import { useSelector, useDispatch } from 'react-redux';  
import type { RootState } from '../store/store';  
import {  
  authenticateAction,  
  authenticatedAction,  
  authorizeAction,  
  authorizedAction,  
} from '../store/userSlice';
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Inside the **Header** component, replace the **useAppContext** call with **useSelector** calls to get the required state:

```
export function Header() {  
  const user = useSelector(  
    (state: RootState) => state.user.user  
  );  
  const loading = useSelector(  
    (state: RootState) => state.user.loading  
  );  
  async function handleSignInClick() {  
    ...  
  }  
  return ...  
}
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Also, call **useDispatch** to get a **dispatch** function:

```
export function Header() {  
  const user = useSelector(  
    (state: RootState) => state.user.user  
  );  
  const loading = useSelector(  
    (state: RootState) => state.user.loading  
  );  
  const dispatch = useDispatch();  
  async function handleSignInClick() {  
    ...  
  }  
  return ...  
}
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - The last thing to do in **Header.tsx** is to modify **handleSignInClick** to reference the action functions from the Redux slice:

```
async function handleSignInClick() {  
  dispatch(authenticateAction());  
  const authenticatedUser = await authenticate();  
  dispatch(authenticatedAction(authenticatedUser));  
  if (authenticatedUser !== undefined) {  
    dispatch(authorizeAction());  
    const authorizedPermissions = await authorize(  
      authenticatedUser.id  
    );  
    dispatch(authorizedAction(authorizedPermissions));  
  }  
}
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Open **Main.tsx** and replace the **AppContext** import statement with import statements for **useSelector** and the **RootState** type:

```
import { useSelector } from 'react-redux';  
import { RootState } from '../store/store';
```



# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Replace the call to **useAppContext** with a call to `useSelector` to get the user state value:

```
export function Main() {  
  const user = useSelector(  
    (state: RootState) => state.user.user  
  );  
  return ...  
}
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Open **Content.tsx** and replace the **AppContext** import statement with import statements for **useSelector** and the **RootState** type:

```
import { useSelector } from 'react-redux';  
import { RootState } from '../store/store';
```

# Using Redux

## Consuming the Redux store in the components

- Carry out the following steps:
  - Replace the call to **useAppContext** with a call to **useSelector** to get the **permissions** state value.
  - Run the app

```
export function Content() {  
  const permissions = useSelector(  
    (state: RootState) => state.user.permissions  
  );  
  if (permissions === undefined) {  
    return null;  
  }  
  return ...  
}
```

# Using Redux

## Recap

- State is stored in a central store
- State is updated by dispatching actions that are handled by reducers
- A **Provider** component needs to be placed appropriately in the component tree to give components access to the Redux store
- Components can select state using a **useSelector** hook and dispatch actions using a **useDispatch** hook



**Q&A**